

Report for Information Aggregator with Web API and Scraping

36122 Python Programmingt

Yan Hao

Yuxiang Wang

Yanzhen Huang

Wenze Qiao

University of Technology Sydney (UTS)

Contents

Contents	1
1 Design Decisions	2
1.1 Module A API	2
1.2 Module B Scraper & Processor	2
1.3 Module C Visulization & GUI	2
1.3.1 Visualization	2
1.3.2 GUI	3
1.4 Module D Main	3
2 Challenges	3
2.1 Module A	3
2.2 Module B	4
2.3 Module C	4
2.4 Module D	4
3 Additional Features	5
3.1 Module A	5
3.2 Module B	5
3.3 Module C	5

1 Design Decisions

1.1 Module A API

1. Object-Oriented Design (OOP)

API requests, parsing, and exception handling are encapsulated in the `NewsAPIClient` class. The module is reusable, extendable, and easy to initialize.

2. Standardized Data Format

Raw API responses are converted into `Article` objects. All internal modules use a unified data structure, enabling clean data layer decoupling.

3. Configuration-Code Separation

Sensitive information such as `API_KEY` and `BASE_URL` is stored in external configuration files, improving security and collaboration efficiency.

1.2 Module B Scraper & Processor

Robust, Layered Field Extraction News websites use highly diverse page structures; therefore, extraction does not rely on a single selector. For each field, the most reliable source is attempted first, followed by fallback mechanisms:

- Title: OpenGraph `og:title`, then `<title>`, then the first `<h1>`.
- Author: `meta[name="author"]`, `meta[property="article:author"]`, byline elements, then `<a rel="author">`.
- Publish date: `meta[property="article:published_time"]` and other common metadata fields, then `<time>` tags.
- Content: semantic `<article>` element first, then all `<p>` tags.

This layered approach ensures the scraper remains robust across various websites without hard-coding site-specific layouts.

A Dedicated DataProcessor Data cleaning and merging operations are separated from scraping logic and implemented in a `DataProcessor` class, which performs three core functions:

- **Cleaning:** Collapse whitespace, convert empty strings to `None`, and flatten nested NewsAPI source objects into plain name strings.
- **Deduplication:** Remove duplicates by URL, then by normalised title (lowercase with punctuation removed), retaining the more complete record.
- **Merging:** Combine API and scraped records by URL, fill missing fields, and retain unmatched scraped records.

1.3 Module C Visualization & GUI

1.3.1 Visualization

1. Constant Configuration Layer

Define unified default labels such as `UNKNOWN_SOURCE` and `UNKNOWN_CATEGORY` to provide global fallback values.

2. General Data Adaptation Layer (Core Utility Functions)

- `article_to_dict()` : A universal data converter compatible with all upstream data formats.
- *Statistics functions* : `count_by_source()`, `count_by_category()`, `count_by_date()` for data aggregation.
- *Cleaning functions* : `_clean_label()`, `_normalise_date()` for data standardisation.

3. Core Chart Building Class

`ArticleChartBuilder` encapsulates all chart plotting logic. It provides three business charts: source distribution, category distribution, and publication timeline. Internal reusable methods include `_bar_chart()` and `_empty_chart()`.

4. Convenient Entry Layer

Top-level functions such as `create_xxx_chart()` allow direct chart generation without class instantiation.

5. Private Auxiliary Layer

Internal methods for label cleaning and date normalisation, hiding underlying implementation details.

1.3.2 GUI

1. Presenter Layer

Pure data presentation logic, GUI-independent, supports independent testing.

2. View Layer

Pure interface rendering, responsible for GUI components construction.

3. Control Layer

Handles interactive logic and connects the view and data layers.

1.4 Module D Main

1. **Central Controller Architecture** Implement a central `NewsAggregatorApp` to initialise, manage and coordinate all sub-modules, achieving low coupling and clear layering.

2. **Modular Single Responsibility Design** Each module performs only its dedicated function (fetching, scraping, processing, visualisation, GUI), ensuring high maintainability.

2 Challenges

2.1 Module A

1. Raw API response is difficult to use directly

- **Challenge:** The API returns unstructured dictionary data with messy fields, which is not suitable for GUI and processing modules.
- **Solution:** Encapsulate data into the internal `Article` object to unify the data format. This decouples the data layer from the business layer and allows upper modules to ignore underlying API structures.

2. Highly duplicated logic between API endpoints

- **Challenge:** `get_top_headlines` and `search_articles` contain repeated code, causing maintenance difficulties.
- **Solution:** Extract common parsing logic into `_parse_articles` for reuse. Both interfaces share the same parsing rules, following the DRY (Don't Repeat Yourself) principle.

3. Security risk of hard-coded API keys and URLs

- **Challenge:** Hard-coded credentials are vulnerable to leakage and difficult to configure across environments.

- **Solution:** Load configurations from a separate `config` module. This separates configuration from code and supports environment switching.

2.2 Module B

- **Inconsistent HTML across websites:** News sites use divergent structures for author and date information. This issue was addressed using the layered fallback strategy instead of fragile fixed selectors.
- **Isolating failures:** A single unreachable URL or malformed page must not disrupt batch processing. Exception handling wraps all requests, returning a `ScrapedArticle` with an error field while allowing the pipeline to continue.
- **Detecting real duplicates:** Similar articles may have slightly different titles or URLs. Deduplication using URL and normalised title significantly improves detection over exact-match methods.
- **Lossless merging:** When API and scraper provide values for the same field, the longer and more informative content is preserved to avoid data loss.
- **Testing without live requests:** Real network calls reduce test speed and reliability. `requests.Session` is mocked to enable fast, deterministic, and repeatable testing.

2.3 Module C

1. Coupling of GUI Logic and Data Presentation Logic

- **Problem:** Tkinter interface code is tightly bound with text formatting and list rendering, making unit testing impossible without opening windows.
- **Solution:** Extract the independent `ArticlePresenter` class for pure data processing without GUI dependencies, enabling testable presentation logic.

2. Poor Compatibility Between Matplotlib and Tkinter

- **Problem:** Matplotlib pops up independent windows by default; repeated plotting causes overlapping components and memory leaks.
- **Solution:** Only render `Figure` objects; implement safe rendering and clearing methods; add exception handling to avoid crashes.

3. Inconsistent Input Data Formats

- **Problem:** Upstream data from API, scraper and processor has inconsistent formats and null value rules.
- **Solution:** Reuse `article_to_dict` for unified data conversion and set default fallback values for all fields.

2.4 Module D

1. Complex Multi-Module Collaboration

- **Problem:** The system contains five independent modules, leading to chaotic invocation and high coupling.
- **Solution:** Adopt a central controller class to unify scheduling and standardise the fixed data processing pipeline.

3 Additional Features

3.1 Module A

- Support multiple news categories and customizable article count.
- Allow users to set the number of articles to fetch.
- Provide two news acquisition methods: top headlines by category and keyword-based search.

3.2 Module B

- Defensive error handling with dedicated error fields to support graceful degradation rather than catastrophic failure.
- Intelligent record merging that preserves the most complete and informative field values instead of blind overwriting.
- Injectable `requests.Session` for simplified testing and built-in support for future caching implementation.

3.3 Module C

- Automated interaction: Auto-select the first article, auto-render charts, auto-wrap long text.
- Flexible control: Customizable article count, 7 news categories, one-click clear function.
- Comprehensive user feedback: Real-time status bar, error prompts, friendly empty chart hints.